# Capstone

March 25, 2026

### 0.0.1 John Behler's Capstone ML Project : Customer Bank Churn

**Data can be found https://www.kaggle.com/datasets/radheshyamkollipara/bank-customer-churn/data**

[2]:
```
# Import libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm

from sklearn import linear_model
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor,␣
↪RandomForestClassifier,GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression, Lasso
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
↪f1_score, confusion_matrix, r2_score, mean_absolute_error,␣
↪mean_squared_error, classification_report, root_mean_squared_error,␣
↪roc_auc_score, RocCurveDisplay
from sklearn.model_selection import train_test_split, GridSearchCV,␣
↪cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler,␣
↪OrdinalEncoder
from sklearn.svm import SVC, SVR

# Load our csv file and name it capstone
capstone = pd.read_csv('Customer-Churn-Records.csv')
```

[3]:
```
# Check the shape of the dataset to see the number of observations and features.
↪ We have 10,000 observations and 18 features
print(capstone.shape)
```

```
(10000, 18)
```

[4]:
```
# Look at the first 5 rows of the dataset to inspect the data to see what data␣
↪we are working with. There is categorical, numerical, and
```

```
# ordinal data. View the column names of our features. See the first three rows␣
↪are not useful to our model so remove them so they do not
# cause noise in the algorithm.
capstone.head()
```

[4]:

```
   RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age  \
0          1    15634602  Hargrave          619    France  Female   42
1          2    15647311      Hill          608     Spain  Female   41
2          3    15619304      Onio          502    France  Female   42
3          4    15701354      Boni          699    France  Female   39
4          5    15737888  Mitchell          850     Spain  Female   43

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
0       2       0.00              1          1               1
1       1   83807.86              1          0               1
2       8  159660.80              3          1               0
3       1       0.00              2          0               0
4       2  125510.82              1          1               1

   EstimatedSalary  Exited  Complain  Satisfaction Score Card Type  \
0        101348.88       1         1                   2   DIAMOND
1        112542.58       0         1                   3   DIAMOND
2        113931.57       1         1                   3   DIAMOND
3         93826.63       0         0                   5      GOLD
4         79084.10       0         0                   5      GOLD

   Point Earned
0           464
1           456
2           377
3           350
4           425
```

[5]:

```
# Look at the values of our categorical variables to see if there are any␣
↪mispellings. None of the data was misspelled. See how many levels
# there are in each category. There are 3 countries in our Geography column, 2␣
↪Genders, and 4 Card Types. We could do this for
# the numerical columns but that is easier to look at with a histograms or␣
↪using the .describe() function to look at the max and min.
print("Unique Values:")
print(capstone['Geography'].unique())
print(capstone['Gender'].unique())
print(capstone['Card Type'].unique())
print(" ")
# Check for duplicated rows. None of our rows are duplicates which is important␣
↪because otherwise we would have to consider deleting the
```

```
# row if it was the same instance because that would have an impact on our␣
↪model.
print("Duplicated Rows:")
print(capstone.duplicated().sum())
```

```
Unique Values:
['France' 'Spain' 'Germany']
['Female' 'Male']
['DIAMOND' 'GOLD' 'SILVER' 'PLATINUM']

Duplicated Rows:
0
```

[6]:
```
# Look at the variables and their datatypes and see if there are any columns␣
↪with null values. None of our data had null values. Look at
# the Dtype and make sure that the columns are appropriate for analysis.␣
↪Ignoring the first 3 features (because we decided those will be
# unnecessary to benefit our model) we have 3 object dtype which is expected␣
↪(Geography, Gender, and Card Type). The rest of our
# data is int64 and float64 which is expected because they are numerical values.
↪
capstone.info()
```

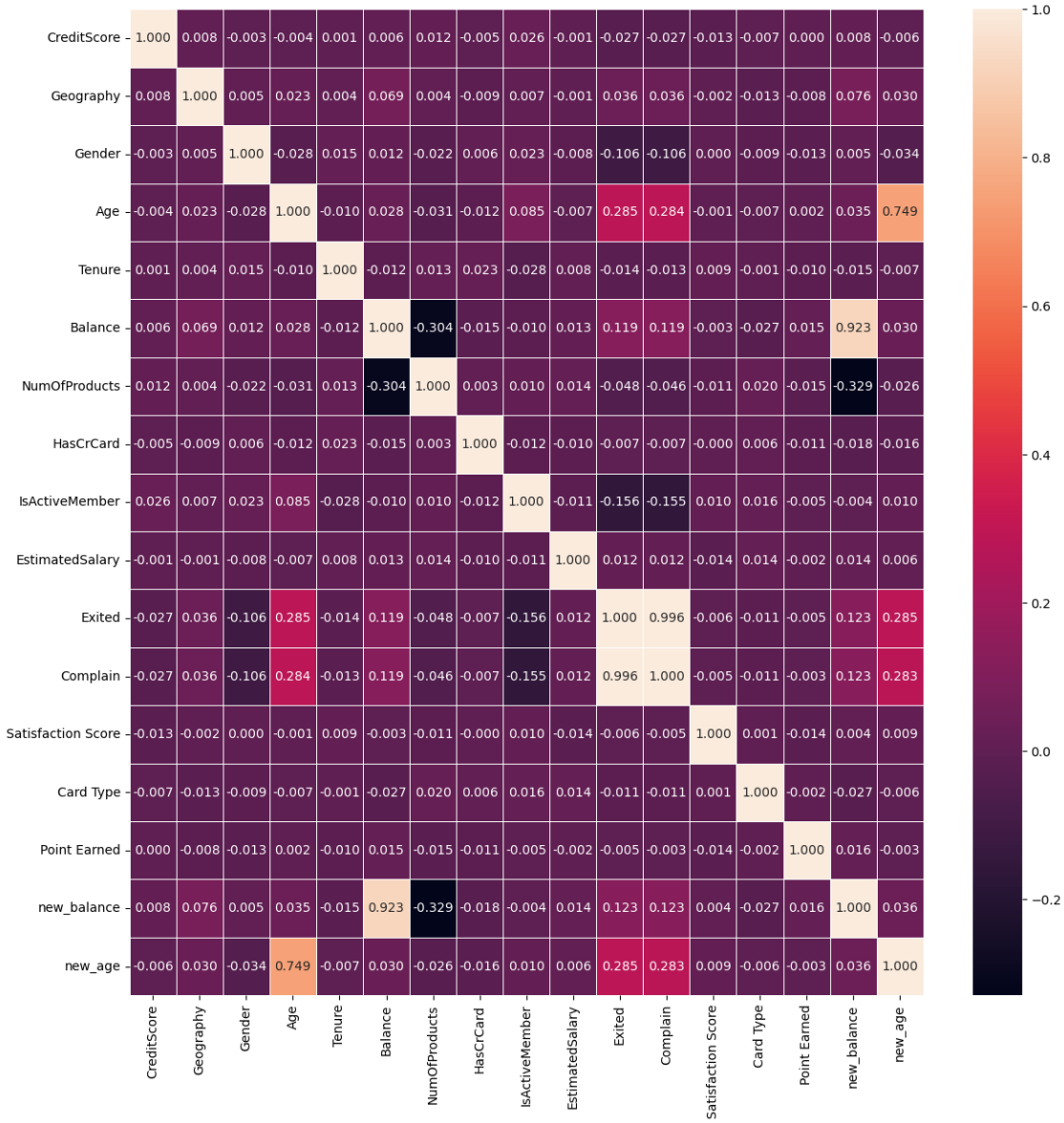
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 18 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   RowNumber           10000 non-null  int64
 1   CustomerId          10000 non-null  int64
 2   Surname             10000 non-null  object
 3   CreditScore         10000 non-null  int64
 4   Geography           10000 non-null  object
 5   Gender              10000 non-null  object
 6   Age                 10000 non-null  int64
 7   Tenure              10000 non-null  int64
 8   Balance             10000 non-null  float64
 9   NumOfProducts       10000 non-null  int64
 10  HasCrCard           10000 non-null  int64
 11  IsActiveMember      10000 non-null  int64
 12  EstimatedSalary     10000 non-null  float64
 13  Exited              10000 non-null  int64
 14  Complain            10000 non-null  int64
 15  Satisfaction Score  10000 non-null  int64
 16  Card Type           10000 non-null  object
 17  Point Earned        10000 non-null  int64
dtypes: float64(2), int64(12), object(4)
memory usage: 1.4+ MB
```

[7]:
```
# Check the statistical summary of the variables in the dataset. This function␣
↪provides the mean, standard deviation, and other basic
# statistical info such as the spread of the data. It is useful to look at the␣
↪minimum and maximum to check for outliers and quickly view
# the range of the data
capstone.describe()
```

[7]:
```
          RowNumber    CustomerId   CreditScore           Age        Tenure  \
count  10000.00000  1.000000e+04  10000.000000  10000.000000  10000.000000
mean    5000.50000  1.569094e+07    650.528800     38.921800      5.012800
std     2886.89568  7.193619e+04     96.653299     10.487806      2.892174
min        1.00000  1.556570e+07    350.000000     18.000000      0.000000
25%     2500.75000  1.562853e+07    584.000000     32.000000      3.000000
50%     5000.50000  1.569074e+07    652.000000     37.000000      5.000000
75%     7500.25000  1.575323e+07    718.000000     44.000000      7.000000
max    10000.00000  1.581569e+07    850.000000     92.000000     10.000000

             Balance  NumOfProducts    HasCrCard  IsActiveMember  \
count   10000.000000   10000.000000  10000.00000    10000.000000
mean    76485.889288       1.530200      0.70550        0.515100
std     62397.405202       0.581654      0.45584        0.499797
min         0.000000       1.000000      0.00000        0.000000
25%         0.000000       1.000000      0.00000        0.000000
50%     97198.540000       1.000000      1.00000        1.000000
75%    127644.240000       2.000000      1.00000        1.000000
max    250898.090000       4.000000      1.00000        1.000000

       EstimatedSalary        Exited      Complain  Satisfaction Score  \
count     10000.000000  10000.000000  10000.000000        10000.000000
mean     100090.239881      0.203800      0.204400            3.013800
std       57510.492818      0.402842      0.403283            1.405919
min          11.580000      0.000000      0.000000            1.000000
25%       51002.110000      0.000000      0.000000            2.000000
50%      100193.915000      0.000000      0.000000            3.000000
75%      149388.247500      0.000000      0.000000            4.000000
max      199992.480000      1.000000      1.000000            5.000000

       Point Earned
count  10000.000000
mean     606.515100
std      225.924839
min      119.000000
25%      410.000000
50%      605.000000
75%      801.000000
max     1000.000000
```

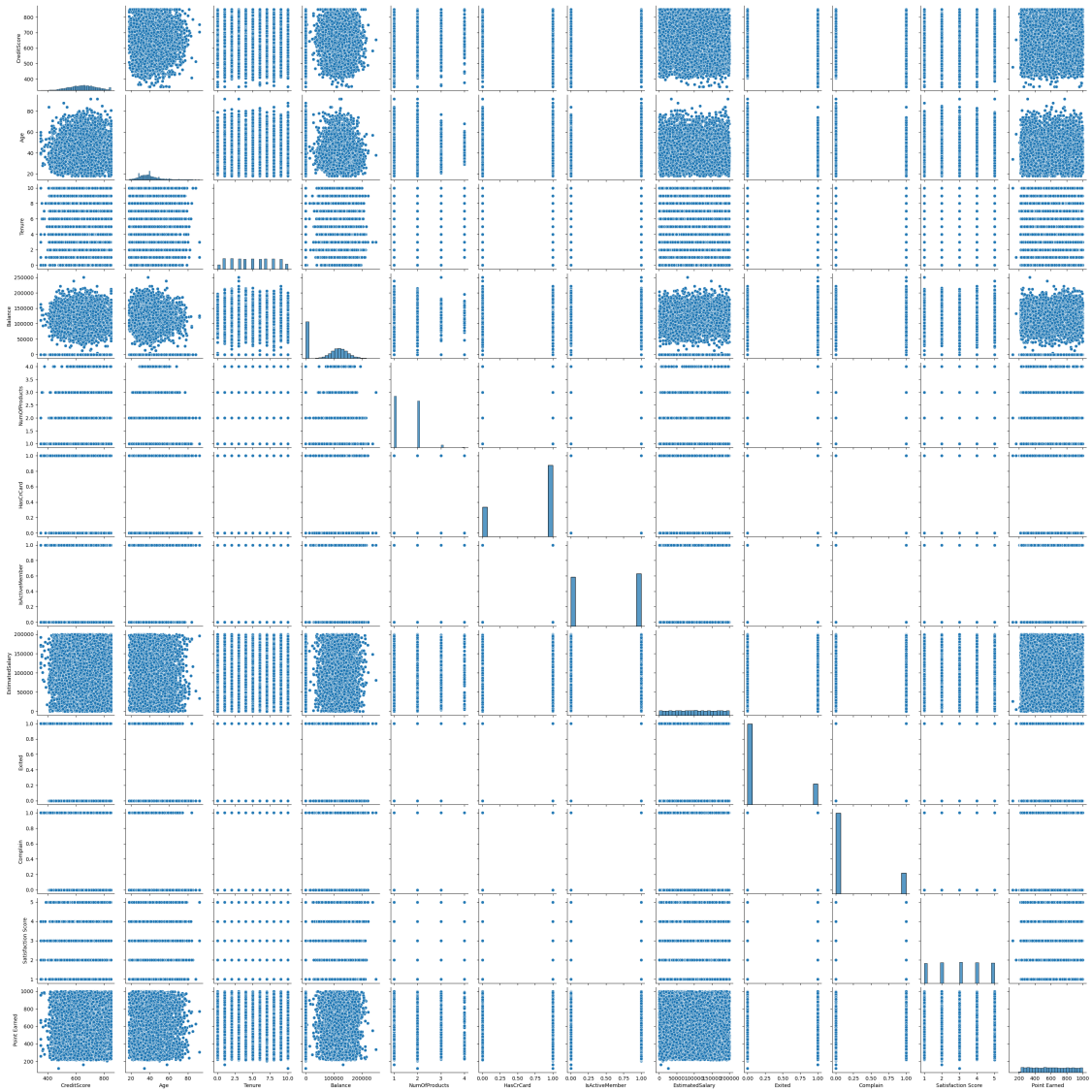
[8]:
```
# Drop columns with meaningless data to our analysis. These features don't have␣
↪any relevance to our goal and will only complicate our
# models and not provide any useful analysis and meaning
capstone = capstone.drop(columns = ['RowNumber', 'CustomerId', 'Surname'])

# Split the features into data types for the column transformer and pipeline.␣
↪Different types of data needs to be preprocessed differently
categorical = ['Geography', 'Gender', 'Card Type']
numerical = ['NumOfProducts','CreditScore', 'Age', 'Balance',␣
↪'EstimatedSalary', 'Point Earned','HasCrCard', 'IsActiveMember', 'Complain']
ordinal = ['Satisfaction Score']
```

[9]:
```
# Make a duplicate of the data so the original dataset doesn't get altered. Its␣
↪always a good idea to have a duplicate of your original
# dataset so you can refer back to it if you need to.
capstone_corr = capstone.copy()

# Encode categorical variables so they can be used for checking correlation and␣
↪modeling. Models can't interpret 'string' data so we must
# convert these to numerical values so they are ready for ML models
encoder = LabelEncoder()
capstone_corr['Geography'] = encoder.fit_transform(capstone_corr['Geography'])
capstone_corr['Gender'] = encoder.fit_transform(capstone_corr['Gender'])
capstone_corr['Card Type'] = encoder.fit_transform(capstone_corr['Card Type'])

# Create a new variable to see if there is any correlation between having a␣
↪balance vs not having a balance. Also create a variable to split
# age into young and old categories. Feature engineer these variable to see if␣
↪they would have any useful relation to our response variable
# "Tenure".
capstone_corr['new_balance'] = (capstone_corr['Balance'] != 0)
capstone_corr['new_age'] = pd.cut(capstone_corr ['Age'], bins = [-1, 37, 94],␣
↪labels =[0, 1])

# Run a correlation heat map to see relationships between features
correlation = capstone_corr.corr()
plt.figure(figsize=(14, 14))
sns.heatmap(correlation, annot= True, fmt=".3f", annot_kws={"size": 10},␣
↪linewidth = 0.7)
plt.show()
```




Since our feature engineered variables "new_balance" and "new_age" had such a low correlations to our response variable, I decided to leave them out of the models even though its possible they would have some correlation with the other variables. We can see that Exited and Complain have a very high correlation which could be an issue for multicollinearity. Age is also somewhat correlated to these variables compared to the rest. Most of our variables have a very low correlation.

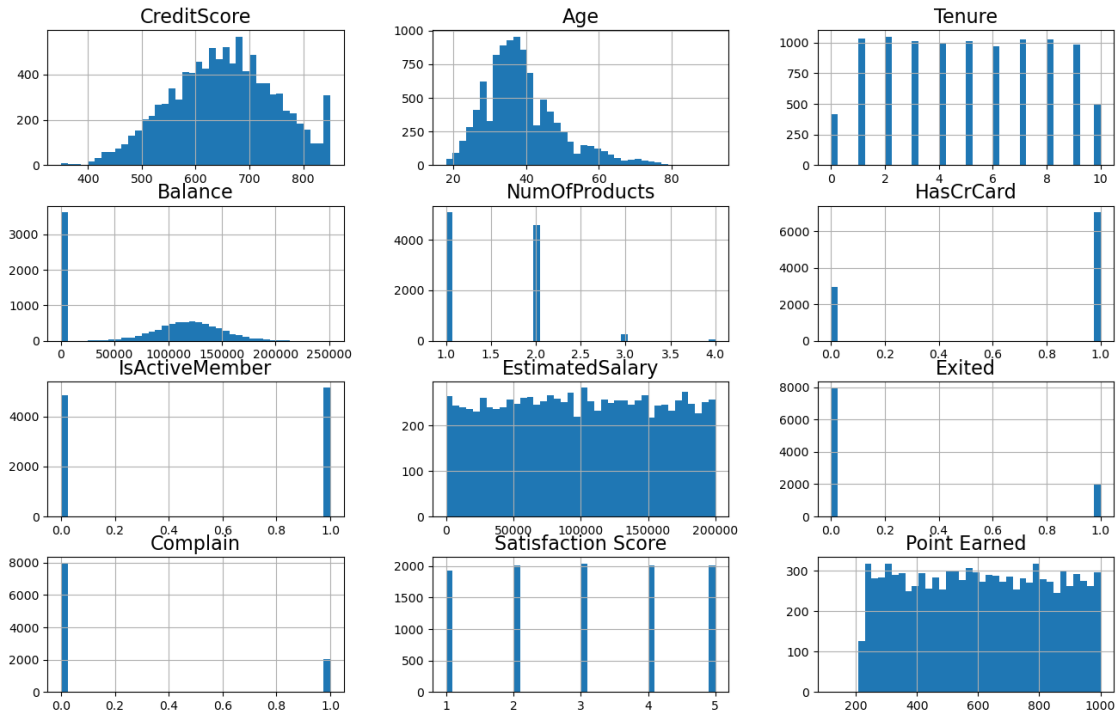
[11]:
```
# Look at a pairplot of the data to see if there were any trends
sns.pairplot(capstone)
plt.show()
```




A lot of our variables are categorical, binary, and uniformly distributed so we don't get much insight from the scatterplots.

[13]:
```
# Create grid parameters
plt.rc('font', size=14)
plt.rc('axes', labelsize=12, titlesize=16)
plt.rc('legend', fontsize=10)
plt.rc('xtick', labelsize=10)
plt.rc('ytick', labelsize=10)

# Plot histograms of numerical and ordinal data so we can look at the␣
↪distribution of the variables
capstone.hist(bins=40, figsize=(16, 10))
```

```
plt.show()
```

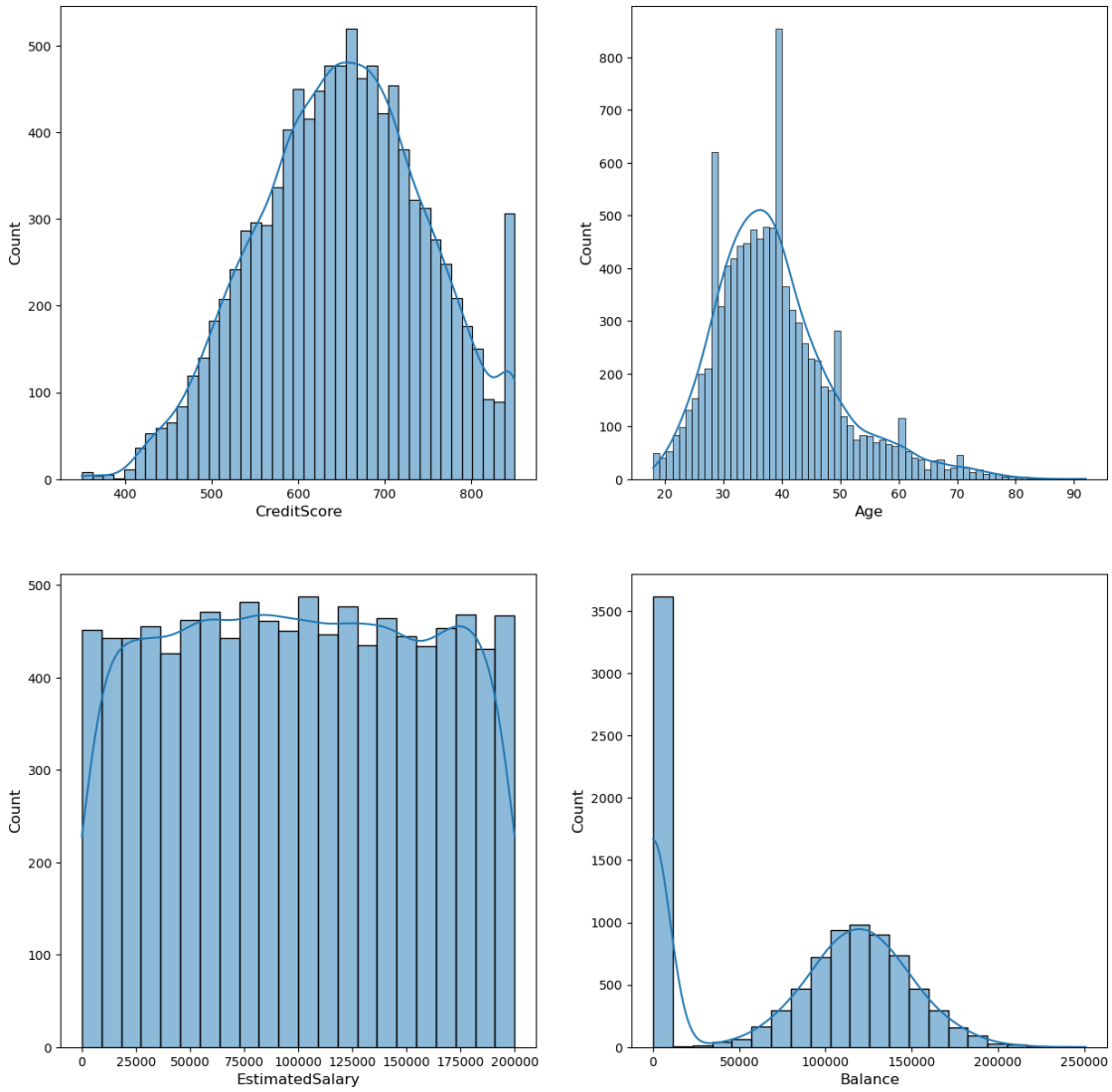



We can see that Credit Score looks normally distributed. Age is right skewed so we will need to transform the data by taking the square root to make it more normally distributed which will help our models perform better. Balance is also normally distributed but has a lot of counts of $0 which is why I already investigated feature engineering a new variable for the correlation matrix but it didn't look promising. Tenure, IsActiveMember, EstimatedSalary, Satistfaction Score, Points Earned were all balanced and distributed equally. NumOfProducts, HasCrCard, Exited, and Complain had some unequal frequencies.

[15]:
```
# Set the parameters for subplots
plt.figure(figsize=(15, 15))

# Create histograms of numerical data using seaborn for better graphics and to␣
↪look at the density curve
plt.subplot(2,2,1)
sns.histplot(data=capstone, x = "CreditScore", kde = True)

plt.subplot(2,2,2)
sns.histplot(data=capstone, x = "Age", kde = True)

plt.subplot(2,2,3)
sns.histplot(data=capstone, x = "EstimatedSalary", kde = True)
```

```
plt.subplot(2,2,4)
sns.histplot(data=capstone, x = "Balance", kde = True)
```

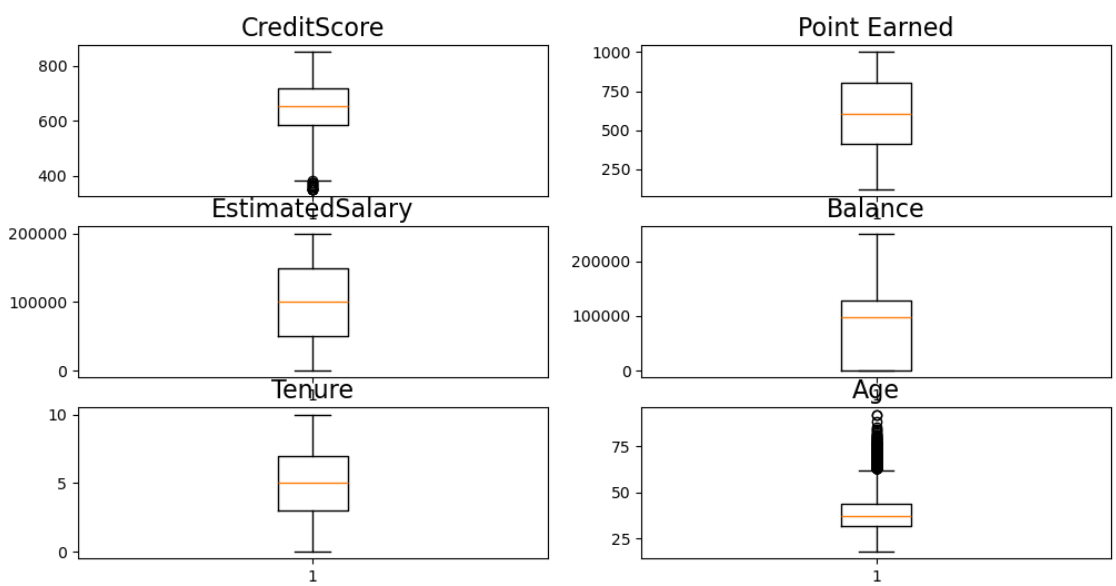
[15]: <Axes: xlabel='Balance', ylabel='Count'>




[16]:
```
# Set the parameters for subplots
plt.figure(figsize =(12, 6))

# Create boxplots of numerical variables so we can check for outliers and see␣
↪if they need to be removed
plt.subplot(3,2,1)
plt.title("CreditScore")
plt.boxplot(capstone["CreditScore"])
```

```
plt.subplot(3,2,2)
plt.title("Point Earned")
plt.boxplot(capstone["Point Earned"])

plt.subplot(3,2,3)
plt.title("EstimatedSalary")
plt.boxplot(capstone["EstimatedSalary"])

plt.subplot(3,2,4)
plt.title("Balance")
plt.boxplot(capstone["Balance"])

plt.subplot(3,2,5)
plt.title("Tenure")
plt.boxplot(capstone["Tenure"])

plt.subplot(3,2,6)
plt.title("Age")
plt.boxplot(capstone["Age"])
```

[16]:
```
{'whiskers': [<matplotlib.lines.Line2D at 0x15a42f5f0>,
  <matplotlib.lines.Line2D at 0x15a42f890>],
 'caps': [<matplotlib.lines.Line2D at 0x15a42fb60>,
  <matplotlib.lines.Line2D at 0x15a42fe30>],
 'boxes': [<matplotlib.lines.Line2D at 0x15a42f320>],
 'medians': [<matplotlib.lines.Line2D at 0x15a42ffe0>],
 'fliers': [<matplotlib.lines.Line2D at 0x15a46c320>],
 'means': []}
```



It appeared the only outliers we have are low credit score and high age so I will keep those because those values are possible and don't seem too extreme. It doesn't appear to be a typo or coded for anything and more of just slight deviation from normal values. We do want to capture all possible variations because those could be important in our models and analysis in tableau

[18]:
```
# Check the distribution between the categorical variables
print("Value Counts:")
print(capstone['Geography'].value_counts())
print(" ")
print(capstone['Gender'].value_counts())
print(" ")
print(capstone['Card Type'].value_counts())

# Notice that feature 'Age' is right skewed so take the square root to make the␣
↪data more normally distributed so it will improve our
# models
capstone['Age'] = np.sqrt(capstone['Age'])

# Set the response variable to Tenure and the rest of the other features as the␣
↪predictors
X = capstone.drop(columns = ['Tenure'])
y = capstone['Tenure']

# Set the random seed to get reproducible code
np.random.seed(42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,␣
↪random_state = 42)

# Create a pipeline for a column transformer. For our numerical and ordinal␣
↪variables, we want to scale the data. And for the categorical
# and ordinal values we want to encode our variables as numerical values
numerical_pipeline = Pipeline([
    ('scaler', StandardScaler())
])

categorical_pipeline = Pipeline([
    ('onehot', OneHotEncoder())
])

ordinal_pipeline = Pipeline([
    ('ordinal', OrdinalEncoder()),
    ('scaler', StandardScaler())
])
```

```
# Create a column transfomer using our pipelines for the features
CT = ColumnTransformer(
    transformers=[
        ('num', numerical_pipeline, numerical),
        ('cat', categorical_pipeline, categorical),
        ('ord', ordinal_pipeline, ordinal),
    ])

# Apply our training and testing sets to the column tranformer to fit and␣
↪transform the data
X_train_CT = CT.fit_transform(X_train)
X_test_CT = CT.fit_transform(X_test)
```

```
Value Counts:
Geography
France     5014
Germany    2509
Spain      2477
Name: count, dtype: int64

Gender
Male      5457
Female    4543
Name: count, dtype: int64

Card Type
DIAMOND     2507
GOLD        2502
SILVER      2496
PLATINUM    2495
Name: count, dtype: int64
```

We can see that the categorical variables are balanced which is important because an inbalance can effect our model.

[20]:
```
# Start off simple by creating a linear regression model using our training␣
↪sets. Train and fit the model using the training data.
linear = linear_model.LinearRegression()
linear.fit(X_train_CT, y_train)

# Predict the response based on our linear regression model using our test set
ytest_predictions = linear.predict(X_test_CT)

# Look at the metrics of our linear regression model. Use cross validation to␣
↪get a model that is more likely to fit to the testing
# data, prevent overfitting, and generalize better to unseen data. Use CV␣
↪throughout the analysis and all models
print("Training Set Metrics Linear Regression Model #1:")
```

```
#print("Model Coefficients:", linear.coef_)
mae_tenure_linear = -cross_val_score(linear, X_train_CT, y_train, cv = 5,␣
 ↪scoring= 'neg_mean_absolute_error')
#print("CV MAE:", mae_tenure_linear)
avg_mae_linear = mae_tenure_linear.mean()
print("Average MAE:", avg_mae_linear)
rmse_tenure_linear = -cross_val_score(linear, X_train_CT, y_train, cv = 5,␣
 ↪scoring= 'neg_root_mean_squared_error')
#print("CV RMSE:", rmse_tenure_linear)
avg_rmse_linear = rmse_tenure_linear.mean()
print("Average RMSE:", avg_rmse_linear)
r2_tenure_linear = -cross_val_score(linear, X_train_CT, y_train, cv = 5,␣
 ↪scoring= 'r2')
#print("CV r^2:", r2_tenure_linear)
avg_r2_linear = r2_tenure_linear.mean()
print("Average r^2:",avg_r2_linear)
print("Standard Deviation RMSE:", rmse_tenure_linear.std())
print(" ")
print("Testing Set Metrics Linear Regression Model #1:")
mae = mean_absolute_error(y_test, ytest_predictions)
print("MAE:", mae)
rmse = np.sqrt(mean_squared_error(y_test, ytest_predictions))
print("RMSE:", rmse)
r2 = r2_score(y_test, ytest_predictions)
print("r^2:", r2)
```

```
Training Set Metrics Linear Regression Model #1:
Average MAE: 2.4850731201171876
Average RMSE: 2.887068358464518
Average r^2: 0.004599150920105233
Standard Deviation RMSE: 0.01685906322687679

Testing Set Metrics Linear Regression Model #1:
MAE: 2.5495859375
RMSE: 2.9319298710157935
r^2: 0.0006899418166018689
```

Our linear regression model didn't produce a good model with weak performance. We see that our data isn't fitting well and most likely isn't linearly correlated. Our r^2 was very poor which means our model is only capturing 0.06% of the the relationship between the predictor variables and response variable. The MAE and RMSE are high relative to our scale of 0-10 so our model is underfitting. Our predicted values were off by about 2.54-2.93 years. This isn't very useful when our range is only 0-10 years (about 25%-30%). This first model suggests that the features in the dataset aren't doing a good job of predicting our response variable linearly.

[22]:
```
# Feature Selection
# Run OLS Regresion to look to see which features had the highest importance␣
 ↪(statistically significant) by looking at the p-values which
```

```python
# indicate week prediction
X_OLS = capstone.drop(columns = ['Tenure'])
y_OLS = capstone['Tenure']

# Encode our categorical data to numerical so they can be interpreted in our␣
↪model
X_OLS['Geography'] = encoder.fit_transform(X_OLS['Geography'])
X_OLS['Gender'] = encoder.fit_transform(X_OLS['Gender'])
X_OLS['Card Type'] = encoder.fit_transform(X_OLS['Card Type'])

# Fit Linear Regression
X = sm.add_constant(X_OLS)
ols_model = sm.OLS(y_OLS, X).fit()

# Print the OLS regression results
print(ols_model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                 Tenure   R-squared:                       0.002
Model:                            OLS   Adj. R-squared:                  0.001
Method:                 Least Squares   F-statistic:                     1.675
Date:                Wed, 25 Mar 2026   Prob (F-statistic):             0.0536
Time:                        20:49:50   Log-Likelihood:                -24797.
No. Observations:               10000   AIC:                         4.962e+04
Df Residuals:                    9985   BIC:                         4.973e+04
Df Model:                          14
Covariance Type:            nonrobust
================================================================================
======
                           coef    std err          t      P>|t|      [0.025
0.975]
--------------------------------------------------------------------------------
------
const                    4.9369      0.344     14.343      0.000       4.262
5.612
CreditScore           4.188e-05      0.000      0.140      0.889      -0.001
0.001
Geography                0.0177      0.035      0.504      0.614      -0.051
0.086
Gender                   0.0808      0.058      1.382      0.167      -0.034
0.195
Age                     -0.0099      0.038     -0.263      0.792      -0.083
0.064
Balance              -3.595e-07   4.91e-07     -0.732      0.464   -1.32e-06
6.03e-07
NumOfProducts            0.0526      0.052      1.007      0.314      -0.050
0.155
```

```
HasCrCard              0.1389      0.063      2.190      0.029       0.015
0.263
IsActiveMember        -0.1786      0.059     -3.023      0.003      -0.294
-0.063
EstimatedSalary      4.03e-07   5.03e-07      0.801      0.423   -5.83e-07
1.39e-06
Exited                -0.5976      0.775     -0.771      0.441      -2.117
0.922
Complain               0.4924      0.774      0.637      0.524      -1.024
2.009
Satisfaction Score     0.0183      0.021      0.891      0.373      -0.022
0.059
Card Type             -0.0040      0.026     -0.157      0.876      -0.055
0.047
Point Earned          -0.0001      0.000     -0.965      0.335      -0.000
0.000
==============================================================================
Omnibus:                     6316.928   Durbin-Watson:                   2.009
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              562.878
Skew:                           0.014   Prob(JB):                    5.92e-123
Kurtosis:                       1.838   Cond. No.                     5.28e+06
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The condition number is large, 5.28e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
```

[23]:
```python
# Get rid of the features that have a low statistical significance (high␣
↪p-values) related to the response variable.
capstone_without_features = capstone.drop(columns = ['CreditScore',␣
↪'Geography', 'Age', 'Balance', 'Complain', 'Card Type'])

# Select the predictor and response variables
X_without_features = capstone_without_features.drop(columns = ['Tenure'])
y_without_features = capstone_without_features['Tenure']

# Drop the variables from our array so they can go through the pipeline/column␣
↪transformer
categorical.remove('Geography')
categorical.remove('Card Type')
numerical.remove('CreditScore')
numerical.remove('Age')
numerical.remove('Balance')
numerical.remove('Complain')
```

```
# Set the random seed to get reproducible code
np.random.seed(42)

# Split the data into training sets and testing sets
X_train_without_features, X_test_without_features, y_train_without_features,␣
↪y_test_without_features = train_test_split(X_without_features,␣
↪y_without_features, test_size = 0.2, random_state = 42)

# Apply our training and testing sets to the column tranformer to fit and␣
↪transform the data
X_train_CT_without_features = CT.fit_transform(X_train_without_features)
X_test_CT_without_features = CT.fit_transform(X_test_without_features)

# Run a new linear regression model after removing the features that don't seem␣
↪statistically significant
linear1 = linear_model.LinearRegression()
linear1.fit(X_train_CT_without_features, y_train_without_features)

# Make prediction based on our test set and our linear regression model
y_pred_without_features = linear1.predict(X_test_CT_without_features)

# Look at the metrics of our 2nd linear regression model
print("Training Set Metrics Linear Model #2:")
mae_tenure_linear1 = -cross_val_score(linear1, X_train_CT_without_features,␣
↪y_train_without_features, cv = 5, scoring= 'neg_mean_absolute_error')
#print("CV MAE:", mae_tenure_linear1)
avg_mae_linear1 = mae_tenure_linear1.mean()
print("Average MAE:", avg_mae_linear1)
rmse_tenure_linear1 = -cross_val_score(linear1, X_train_CT_without_features,␣
↪y_train_without_features, cv = 5, scoring= 'neg_root_mean_squared_error')
#print("CV RMSE:", rmse_tenure_linear1)
avg_rmse_linear1 = rmse_tenure_linear1.mean()
print("Average RMSE:", avg_rmse_linear1)
r2_tenure_linear1 = -cross_val_score(linear1, X_train_CT_without_features,␣
↪y_train_without_features, cv = 5, scoring= 'r2')
#print("CV r^2:", r2_tenure_linear1)
avg_r2_linear1 = r2_tenure_linear1.mean()
print("Average r^2:",avg_r2_linear1)
print("Standard Deviation RMSE:", rmse_tenure_linear1.std())
print(" ")
print("Testing Set Metrics Linear Model #2:")
mae_without_features = mean_absolute_error(y_test_without_features,␣
↪y_pred_without_features)
print("MAE:",mae_without_features)
rmse_without_features = np.sqrt(mean_squared_error(y_test_without_features,␣
↪y_pred_without_features))
```

```
print("RMSE:",rmse_without_features)
r2_without_features = r2_score(y_test_without_features, y_pred_without_features)
print("r^2:",r2_without_features)
```

```
Training Set Metrics Linear Model #2:
Average MAE: 2.4795245418173737
Average RMSE: 2.8814759526394886
Average r^2: 0.0007065373718723844
Standard Deviation RMSE: 0.0174129751256103

Testing Set Metrics Linear Model #2:
MAE: 2.549591796875
RMSE: 2.933256619562925
r^2: -0.00021467269144692658
```

This model is very poor with slightly worse testing set metrics of MAE, RMSE and R^2. Even after removing features with high p-values to simply the model, our predictability actually suffered. This means some variables may interact to influence the response.

[25]:
```
# Since the model above did not improve and actually looked slightly worse use␣
↪the initial dataset with all of our features and run a lasso
# regression model. This will penalize and shrink the coefficients of less␣
↪importance and is used when there are many features. According to
# our correlation matrix, there may be features that are irrelevant to␣
↪predicting tenure. Lasso helps to apply this regularization and
# identify features that are not valuable to our model.
lasso_model = Lasso()

# Set the random seed to get reproducible code
np.random.seed(42)

# Fit the training data to our lasso model
lasso_model.fit(X_train_CT, y_train)

# Make prediction based on our test set and our lasso regression model
lasso_predictions = lasso_model.predict(X_test_CT)

# Look at the metrics of our lasso regression model
print("Training Set Metrics Lasso Model:")
print("Coefficients :", lasso_model.coef_)
mae_tenure_lasso = -cross_val_score(lasso_model, X_train_CT, y_train, cv = 5,␣
↪scoring= 'neg_mean_absolute_error')
#print("CV MAE:", mae_tenure_lasso)
avg_mae_lasso = mae_tenure_lasso.mean()
print("Average MAE:", avg_mae_lasso)
rmse_tenure_lasso = -cross_val_score(lasso_model, X_train_CT, y_train, cv = 5,␣
↪scoring= 'neg_root_mean_squared_error')
#print("CV RMSE:", rmse_tenure_lasso)
```

```
avg_rmse_lasso = rmse_tenure_lasso.mean()
print("Average RMSE:", avg_rmse_lasso)
r2_tenure_lasso = -cross_val_score(lasso_model, X_train_CT, y_train, cv = 5,␣
 ↪scoring= 'r2')
#print("CV r^2:", r2_tenure_lasso)
avg_r2_lasso = r2_tenure_lasso.mean()
print("Average r^2:",avg_r2_lasso)
print("Standard Deviation RMSE:", rmse_tenure_lasso.std())
print(" ")
print("Testing Set Metrics Lasso Model:")
mae_lasso = mean_absolute_error(y_test, lasso_predictions)
print("MAE:", mae_lasso)
rmse_lasso = root_mean_squared_error(y_test, lasso_predictions)
print("RMSE:", rmse_lasso)
r2_lasso = r2_score(y_test, lasso_predictions)
print("r^2:", r2_lasso)
```

```
Training Set Metrics Lasso Model:
Coefficients : [ 0. -0. -0. -0.  0. -0.  0. -0. -0. -0. -0.  0. -0.  0. -0.  0.
-0.  0.
  0.]
Average MAE: 2.4718399218749996
Average RMSE: 2.88223635659311
Average r^2: 0.0012419994174632353
Standard Deviation RMSE: 0.015499205072357912

Testing Set Metrics Lasso Model:
MAE: 2.542813875
RMSE: 2.9332812924479303
r^2: -0.00023149923517640225
```

Our MAE did slightly improve but our RMSE and r^2 worsened. Once again we have poor model metrics which suggests there might not be linearity in our data. We can see the coefficients of our lasso model are all zero, so the model found no features to be significant in predicting 'tenure'.

[27]:
```
# Created an SVR model because it is more flexible and to look for non-linear␣
 ↪pattern. SVR is good at capturing complex trends in our data
# that linear regression models cannot. Our linear regression models didn't␣
 ↪provide good results and maybe were too simplistic.
svr_reg = SVR()

# Set the random seed to get reproducible code
np.random.seed(42)

# Fit the training data
svr_reg.fit(X_train_CT, y_train)

# Made prediction based on our test set and our SVR regression model
```

```
SVR_predictions = svr_reg.predict(X_test_CT)

# Looked at the metrics of our SVR model
print("Training Set Metrics SVR Model:")
mae_tenure_svr = -cross_val_score(svr_reg, X_train_CT, y_train, cv = 5,␣
 ↪scoring= 'neg_mean_absolute_error')
#print("CV MAE:", mae_tenure_svr)
avg_mae = mae_tenure_svr.mean()
print("Average MAE:", avg_mae)
rmse_tenure_svr = -cross_val_score(svr_reg, X_train_CT, y_train, cv = 5,␣
 ↪scoring= 'neg_root_mean_squared_error')
#print("CV RMSE:", rmse_tenure_svr)
avg_rmse = rmse_tenure_svr.mean()
print("Average RMSE:", avg_rmse)
r2_tenure_svr = -cross_val_score(svr_reg, X_train_CT, y_train, cv = 5, scoring=␣
 ↪'r2')
#print("CV r^2:", r2_tenure_svr)
avg_r2 = r2_tenure_svr.mean()
print("Average r^2:",avg_r2)
print("Standard Deviation RMSE:", rmse_tenure_svr.std())
print(" ")
print("Testing Set Metrics SVR Model:")
mae = mean_absolute_error(y_test, SVR_predictions)
print("MAE:", mae)
rmse = root_mean_squared_error(y_test, SVR_predictions)
print("RMSE:", rmse)
r2 = r2_score(y_test, SVR_predictions)
print("r^2:", r2)
```

```
Training Set Metrics SVR Model:
Average MAE: 2.5277780145075783
Average RMSE: 2.946217167912507
Average r^2: 0.04616653070054859
Standard Deviation RMSE: 0.02891629177498457

Testing Set Metrics SVR Model:
MAE: 2.599626208716978
RMSE: 3.0055575654349904
r^2: -0.050130332758293594
```

We have another poor model and still have not seen any metrics that looks promising for a good model. Our SVR Model had the worst metrics of our models so far which might indicate that our data might be the primary issue. I will continue to check out a few other models to see if they can handle the complexity of the data.

[29]:
```
# Create a random forest regression model which is a tree based algorithm to␣
 ↪capture feature interactions and reduce overfitting to improve
# prediction accuracy
```

```python
rfmodel = RandomForestRegressor()

# Set the random seed to get reproducible code
np.random.seed(42)

# Fit the training data
rfmodel.fit(X_train_CT, y_train)

# Made prediction based on our test set and our random forest regression model
predicted = rfmodel.predict(X_test_CT)

# Look at the metrics of our random forest regression model
print("Training Set Metrics Random Forest Model:")
mae_tenure_rf = -cross_val_score(rfmodel, X_train_CT, y_train, cv = 5, scoring=␣
↪'neg_mean_absolute_error')
#print("CV MAE Random Forest:", mae_tenure_rf)
avg_mae_rf = mae_tenure_rf.mean()
print("Average MAE:", avg_mae_rf)
rmse_tenure_rf = -cross_val_score(rfmodel, X_train_CT, y_train, cv = 5,␣
↪scoring= 'neg_root_mean_squared_error')
#print("CV RMSE Random Forest:", rmse_tenure_rf)
avg_rmse_rf = rmse_tenure_rf.mean()
print("Average RMSE:", avg_rmse_rf)
r2_tenure_rf = -cross_val_score(rfmodel, X_train_CT, y_train, cv = 5, scoring=␣
↪'r2')
#print("CV r^2 Random Forest:", r2_tenure_rf)
avg_r2_rf = r2_tenure_rf.mean()
print("Average r^2:",avg_r2_rf)
print("Standard Deviation RMSE:", rmse_tenure_rf.std())
print(" ")
print("Testing Set Metrics Random Forest Model:")
mae = mean_absolute_error(y_test, predicted)
print("MAE:", mae)
rmse = root_mean_squared_error(y_test, predicted)
print("RMSE:", rmse)
r2 = r2_score(y_test, predicted)
print("r^2:", r2)
```

```
Training Set Metrics Random Forest Model:
Average MAE: 2.5152537500000003
Average RMSE: 2.923513476024942
Average r^2: 0.03351388998258002
Standard Deviation RMSE: 0.015952053176928296

Testing Set Metrics Random Forest Model:
MAE: 2.5844099999999997
RMSE: 2.9868973367024183
```

```
r^2: -0.03713118622032496
```

We still have a poor model. MAE got worse but RMSE and r^2 did perform better than our SVM Model but worse than our linear regression model. Lastly I will try gradient boosting and if that doesn't come up with any good results, I will look into changing our response variable.

[31]:
```
# Create a gradient boosting regression model which is a tree based model to␣
↪look for nonlinearities and complex patterns
gbr = GradientBoostingRegressor()

# Set the random seed to get reproducible code
np.random.seed(42)

# Fit the training data to our gradient boosting regression model
gbr.fit(X_train_CT, y_train)

# Make prediction based on our test set and our gradient boosting regression␣
↪model
gbr_predictions = gbr.predict(X_test_CT)

# Look at the metrics of our gradient boosting regression model. Our model␣
↪still is very poor with no improvements
print("Training Set Metrics Gradient Boosting Model:")
mae_tenure_gbr = -cross_val_score(gbr, X_train_CT, y_train, cv = 5, scoring=␣
↪'neg_mean_absolute_error')
#print("CV MAE:", mae_tenure_gbr)
avg_mae_gbr = mae_tenure_gbr.mean()
print("Average MAE:", avg_mae_gbr)
rmse_tenure_gbr = -cross_val_score(gbr, X_train_CT, y_train, cv = 5, scoring=␣
↪'neg_root_mean_squared_error')
#print("CV RMSE:", rmse_tenure_gbr)
avg_rmse_gbr = rmse_tenure_gbr.mean()
print("Average RMSE:", avg_rmse_gbr)
r2_tenure_gbr = -cross_val_score(gbr, X_train_CT, y_train, cv = 5, scoring=␣
↪'r2')
#print("CV r^2:", r2_tenure_gbr)
avg_r2_gbr = r2_tenure_gbr.mean()
print("Average r^2:",avg_r2_gbr)
print("Standard Deviation RMSE:", rmse_tenure_gbr.std())
print(" ")
print("Testing Set Metrics Gradient Boosting Model:")
mae = mean_absolute_error(y_test, gbr_predictions)
print("MAE:", mae)
rmse = root_mean_squared_error(y_test, gbr_predictions)
print("RMSE:", rmse)
r2 = r2_score(y_test, gbr_predictions)
print("r^2:", r2)
```

```
Training Set Metrics Gradient Boosting Model:
Average MAE: 2.4964789602245263
Average RMSE: 2.900790262532547
Average r^2: 0.013996596004549921
Standard Deviation RMSE: 0.020328300491716272

Testing Set Metrics Gradient Boosting Model:
MAE: 2.5712407455680446
RMSE: 2.953371443206134
r^2: -0.013979663572447176
```

None of our models have been good. Our data is most likely very noisy. Our original linear regression model performed the best of all of our models, however, given the two linear regression models and a lasso model with all of our coefficients 0, it appears there is no linear relationship between our data because none of then have improved our model significantly. Of the other models, the gradient boosting model performed better than the random forest and SVM model, so I decided to do a grid search with the GBR model. Hopefully the grid search will understand the more complex data and minimize errors. A grid search will optimize the hyperparameters to create a better model to be able to generalize to unseen data even though its unlikely to significantly give us better results.

[ ]:
```
# Set the parameter grid for a grid search
#pg_gbr = {
#    'alpha': [0.001, 0.01],
#    'learning_rate': [0.01, 0.05],
#    'min_samples_leaf': [1, 3]
#}

# Set the random seed to get reproducible code
np.random.seed(42)

pg_gbr = {
    'n_estimators': [150, 250, 350],
    'max_depth': [2, 4, 6],
}

# Run the grid search on our gradient boosting regression model with the␣
↪parameter grid
grid_search = GridSearchCV(gbr, pg_gbr, cv = 5)

# Fit the grid seach with the training data
grid_search.fit(X_train_CT, y_train)

# Best Parameters
best_params = grid_search.best_params_
print("Best Parameter:", best_params)

# Best Model
```

```
bestscore = grid_search.best_score_
print("Best Score:", bestscore)
bestmodel = grid_search.best_estimator_
print("Best Score:", bestmodel)

# Look at the metrics of our grid search of the GBR model
gridsearch_predictions = bestmodel.predict(X_test_CT)
mae = mean_absolute_error(y_test, gridsearch_predictions)
print("Testing Set Metrics Grid Search:")
print("MAE:", mae)
rmse = root_mean_squared_error(y_test, gridsearch_predictions)
print("RMSE:", rmse)
r2 = r2_score(y_test, gridsearch_predictions)
print("r^2:", r2)
```

As we can see, our parameter grid did give us better results, however, it wasn't significant which was expected. Once again, it seems our data does not have any relationships between the predictors and the continous response variable. Lets try to bin 'Tenure' and see if there is any way to predict long vs short term customers.

[ ]:
```
# Make a copy of the dataset so the original data doesn't get altered
capstone_tenure = capstone.copy()

# Since the age feature is right skewed take the square root to make the data␣
↪more normally distributed
capstone_tenure['Age'] = np.sqrt(capstone_tenure['Age'])

# Bin the Tenure Variable so if they were at the bank from 0-5 years it will be␣
↪coded 0 for "short term" and 6-10 will be 1 for "long term"
capstone_tenure['Tenure'] = pd.cut(capstone_tenure['Tenure'], bins=[-1, 5, 10],␣
↪labels=[0, 1])

# Set the response variable to the binned variable "Tenure" and the other␣
↪features as the predictors
X_tenure = capstone_tenure.drop(columns = ['Tenure'])
y_tenure = capstone_tenure['Tenure']

# Set the random seed to get reproducible code
np.random.seed(42)

# Split the data into training sets and testing sets
X_train_tenure, X_test_tenure, y_train_tenure, y_test_tenure =␣
↪train_test_split(X_tenure, y_tenure, test_size = 0.2, random_state = 42)

# Apply our training and testing sets to the column tranformer to fit and␣
↪transform the data
X_train_CT_tenure = CT.fit_transform(X_train_tenure)
```

```
X_test_CT_tenure = CT.fit_transform(X_test_tenure)

# Check the value counts to make sure the data was balanced
print(capstone_tenure['Tenure'].value_counts())
print(" ")

# Create a logistic regression and used class_weight = 'balanced' to adjust for␣
↪the class imbalance
logistic = LogisticRegression(class_weight='balanced')
logistic.fit(X_train_CT_tenure, y_train_tenure)

# Make prediction based on our test set and our linear regression model
y_pred_tenure = logistic.predict(X_test_CT_tenure)

# Look at the metrics of our logistic regression model. Once again we have a␣
↪poor model
accuracy = accuracy_score(y_test_tenure, y_pred_tenure)
print("Testing Set Metrics Logistic Regression:")
print("Accuracy:", accuracy)
print("Precision:", precision_score(y_test_tenure, y_pred_tenure,␣
↪zero_division= 0))
print("Recall:", recall_score(y_test_tenure, y_pred_tenure))
print("F1 Score:", f1_score(y_test_tenure, y_pred_tenure))
print("Confusion Matrix:", confusion_matrix(y_test_tenure, y_pred_tenure))
```

This model is accurate just slightly over 50% which slightly better than randomly guessing but given the low precision, and f1 score this is not a good model. Without fixing the class imbalancing, our model previously predicted mainly one class with very low recall, f1, and precision.

[ ]:
```
# Make a copy of our data so our original dataset isn't affected
capstone_left = capstone.copy()

# Delete the rows where the customer hadn't left the bank yet. If they haven't␣
↪left yet then maybe those customers that are still at the
# bank are affecting our analysis.
capstone_left = capstone_left[capstone_left['Exited'] == 1]

# Split the features into data types for the column transformer
categorical_left = ['Geography', 'Gender', 'Card Type']
numerical_left = ['NumOfProducts','CreditScore', 'Age', 'Balance',␣
↪'EstimatedSalary', 'Point Earned','HasCrCard', 'IsActiveMember', 'Complain']
ordinal_left = ['Satisfaction Score']

# Set the response variable to Tenure and the other features as the predictors
X_left = capstone_left.drop(columns = ['Tenure', 'Exited'])
y_left = capstone_left['Tenure']
```

```
# Set the random seed to get reproducible code
np.random.seed(42)

# Split the data into training sets and testing sets
X_train_left, X_test_left, y_train_left, y_test_left = train_test_split(X_left,␣
↪y_left, test_size = 0.2, random_state = 42)

# Create a column transfomer using our pipeline for the features
CT1 = ColumnTransformer(
    transformers=[
        ('num', numerical_pipeline, numerical_left),
        ('cat', categorical_pipeline, categorical_left),
        ('ord', ordinal_pipeline, ordinal_left),
    ])

# Apply our training and testing sets to the column tranformer to fit and␣
↪transform the data
X_train_CT_left = CT1.fit_transform(X_train_left)
X_test_CT_left = CT1.fit_transform(X_test_left)

# Run a linear regression on just the customers who had left the bank
model_left = linear_model.LinearRegression()
model_left.fit(X_train_CT_left, y_train_left)

# Look at the metrics of our linear regression model
print("Training Set Metrics Linear Regression Model #3:")
mae_left = -cross_val_score(model_left, X_train_CT_left, y_train_left, cv = 5,␣
↪scoring= 'neg_mean_absolute_error')
#print("CV MAE:", mae_left)
avg_mae = mae_left.mean()
print("Average MAE:", avg_mae)
rmse_left = -cross_val_score(model_left, X_train_CT_left, y_train_left, cv = 5,␣
↪scoring= 'neg_root_mean_squared_error')
#print("CV RMSE:", rmse_left)
avg_rmse = rmse_left.mean()
print("Average RMSE:", avg_rmse)
r2_left = -cross_val_score(model_left, X_train_CT_left, y_train_left, cv = 5,␣
↪scoring= 'r2')
#print("CV r^2:", r2_left)
avg_r2 = r2_left.mean()
print("Average r^2:",avg_r2)
print("Standard Deviation RMSE:", rmse_left.std())

print(" ")
print("Testing Set Metrics Linear Regression Model #3:")
y_pred_left = model_left.predict(X_test_CT_left)
mae_left = mean_absolute_error(y_test_left, y_pred_left)
```

```
print("MAE:", mae_left)
rmse_left = np.sqrt(mean_squared_error(y_test_left, y_pred_left))
print("RMSE:", rmse_left)
r2_left = r2_score(y_test_left, y_pred_left)
print("r^2:", r2_left)
```

Our model performed just as bad so after trying multiple different things, I have come to the conclusion that from our data it is very hard to predict tenure from our features even after trying multiple different options. Our models haven't accomplished predictive performance, which makes sense because of weak feature correlation. One suggestion would be to include other features and new customer demographics in the dataset to see if there is something else we can use to predict the number of years a customer will stay at the bank. Lets take a look at a new feature as the response variable. We have data on if the customer has left the bank or not, so lets look to implement a model that can predict whether a customer will leave or not.

[ ]:
```
# Create a copy of the dataset
capstone_exited = capstone.copy()

# Split the features into data types for the column transformer
categorical_exited = ['Geography', 'Gender', 'Card Type']
numerical_exited = ['NumOfProducts','CreditScore', 'Age', 'Balance',␣
↪'EstimatedSalary', 'Point Earned','HasCrCard', 'IsActiveMember', 'Complain',␣
↪'Tenure']
ordinal_exited = ['Satisfaction Score']

# Changed the response variable to Exited
X_exited = capstone_exited.drop(columns = ['Exited'])
y_exited = capstone_exited['Exited']

# Set the random seed to get reproducible code
np.random.seed(42)

# Split the data into training sets and testing sets
X_train_exited, X_test_exited, y_train_exited, y_test_exited =␣
↪train_test_split(X_exited, y_exited, test_size = 0.2, random_state = 42)

# Look at the value counts of the capstone exited
capstone_exited['Exited'].value_counts()

# Create a column transfomer using our pipeline for the features
CT2 = ColumnTransformer(
    transformers=[
        ('num', numerical_pipeline, numerical_exited),
        ('cat', categorical_pipeline, categorical_exited),
        ('ord', ordinal_pipeline, ordinal_exited),
    ])
```

```
# Apply our training and testing sets to the column tranformer to fit and␣
↪transform the data
X_train_CT_exited = CT2.fit_transform(X_train_exited)
X_test_CT_exited = CT2.fit_transform(X_test_exited)

# Run a Logistic Regression model
model_exited = LogisticRegression(class_weight='balanced')
model_exited.fit(X_train_CT_exited, y_train_exited)

# Print the metrics from the logistic regression model
y_pred_exited = model_exited.predict(X_test_CT_exited)
accuracy_exited = accuracy_score(y_test_exited, y_pred_exited)

# Look at the metrics of our logistic regression model
print("Testing Set Metrics Logistic Regression:")
print("Accuracy:", accuracy_exited)
print("Precision:", precision_score(y_test_exited, y_pred_exited,␣
↪zero_division= 0))
print("Recall:", recall_score(y_test_exited, y_pred_exited))
print("F1 Score:", f1_score(y_test_exited, y_pred_exited))
print("Confusion Matrix:", confusion_matrix(y_test_exited, y_pred_exited))

svc_disp = RocCurveDisplay.from_estimator(model_exited, X_test_CT_exited,␣
↪y_test_exited)
plt.show()
```

This is a very good model. Very high acuracy, precision, recall, and f1 score all over 99.9%. Lets explore other models to see if we can still increase our metrics.

[ ]:
```
# Random Forest, SVC, and Gradient Boosting Regression Model with Exited as the␣
↪response variable

# Set the random seed to get reproducible code
np.random.seed(42)

# Run a Random Forest Classifier model
randforest = RandomForestClassifier()
randforest.fit(X_train_CT_exited, y_train_exited)
y_pred_randforest = randforest.predict(X_test_CT_exited)

# Look at the metrics of our random forest classifer model
accuracy_randforest = accuracy_score(y_test_exited, y_pred_randforest)
y_prob = randforest.predict_proba(X_test_CT_exited)[:, 1]
roc_auc = roc_auc_score(y_test_exited, y_prob)
print("Testing Set Metrics Random Forest Model:")
print("ROC AUC score:", roc_auc)
print("Accuracy:", accuracy_randforest)
```

```
print("Precision:", precision_score(y_test_exited, y_pred_randforest,␣
↪zero_division= 0))
print("Recall:", recall_score(y_test_exited, y_pred_randforest))
print("F1 Score:", f1_score(y_test_exited, y_pred_randforest))
print("Confusion Matrix:", confusion_matrix(y_test_exited, y_pred_randforest))
print(" ")

# Run a SVC model and printed the metrics
SVC_exited = SVC(class_weight='balanced')
SVC_exited.fit(X_train_CT_exited, y_train_exited)
y_pred_SVC = SVC_exited.predict(X_test_CT_exited)

# Look at the metrics of our SVC regression model
accuracy_SVC = accuracy_score(y_test_exited, y_pred_SVC)
print("Testing Set Metrics SVC Model:")
print("Accuracy:", accuracy_SVC)
print("Precision:", precision_score(y_test_exited, y_pred_SVC, zero_division=␣
↪0))
print("Recall:", recall_score(y_test_exited, y_pred_SVC))
print("F1 Score:", f1_score(y_test_exited, y_pred_SVC))
print("Confusion Matrix:", confusion_matrix(y_test_exited, y_pred_SVC))
print(" ")

# Run a Gradient Boosting Classifer model
GBR_exited = GradientBoostingClassifier()
GBR_exited.fit(X_train_CT_exited, y_train_exited)
y_pred_GBR = GBR_exited.predict(X_test_CT_exited)

# Look at the metrics of our gradient boosting classifier model
accuracy_GBR = accuracy_score(y_test_exited, y_pred_GBR)
y_prob1 = GBR_exited.predict_proba(X_test_CT_exited)[:, 1]
roc_auc1 = roc_auc_score(y_test_exited, y_prob1)
print("Testing Set Metrics Gradient Boosting Model:")
print("ROC AUC score:", roc_auc1)
print("Accuracy:", accuracy_GBR)
print("Precision:", precision_score(y_test_exited, y_pred_GBR, zero_division=␣
↪0))
print("Recall:", recall_score(y_test_exited, y_pred_GBR))
print("F1 Score:", f1_score(y_test_exited, y_pred_GBR))
print("Confusion Matrix:", confusion_matrix(y_test_exited, y_pred_GBR))
```

The random forest classifer model and SVC model gave us the same metrics as logistic regression model. Our gradient boosting regression model performed the worst. Lets take a look at models without our Complain feature as that is likely influencing how good our models are because it has a 0.996 correlation with our response variable.

[ ]:
```
# Logistic, SVC, Random Forest, and Gradient Boosting Regression Model without␣
↪Complain variable and Exited as the response variable

# Create a copy of the dataset
capstone_complain = capstone.copy()
capstone_complain = capstone_complain.drop(columns = ['Complain'])

# Split the features into data types for the column transformer
categorical_complain = ['Geography', 'Gender', 'Card Type']
numerical_complain = ['NumOfProducts','CreditScore', 'Age', 'Balance',␣
↪'EstimatedSalary', 'Point Earned','HasCrCard', 'IsActiveMember', 'Tenure']
ordinal_complain = ['Satisfaction Score']

# Set the response variable to Exited
X_complain = capstone_complain.drop(columns = ['Exited'])
y_complain = capstone_complain['Exited']

# Set the random seed to get reproducible code
np.random.seed(42)

# Split the data into training sets and testing sets
X_train_complain, X_test_complain, y_train_complain, y_test_complain =␣
↪train_test_split(X_complain, y_complain, test_size = 0.2, random_state = 42)

# Column Transfomer
CT3 = ColumnTransformer(
    transformers=[
        ('num', numerical_pipeline, numerical_complain),
        ('cat', categorical_pipeline, categorical_complain),
        ('ord', ordinal_pipeline, ordinal_complain),
    ])

# Apply our training and testing sets to the column tranformer to fit and␣
↪transform the data
X_train_CT_com = CT3.fit_transform(X_train_complain)
X_test_CT_com = CT3.fit_transform(X_test_complain)

# Logistic Regression
logistic_complain = LogisticRegression(class_weight='balanced')
logistic_complain.fit(X_train_CT_com, y_train_complain)

# Made prediction based on our test set and our linear regression model
y_pred_com = logistic_complain.predict(X_test_CT_com)

# Looked at the metrics of our logistic regression model
accuracy = accuracy_score(y_test_complain, y_pred_com)
print("Testing Set Metrics Logistic Regression:")
```

```
print("Accuracy:", accuracy)
print("Precision:", precision_score(y_test_complain, y_pred_com, zero_division=␣
↪0))
print("Recall:", recall_score(y_test_complain, y_pred_com))
print("F1 Score:", f1_score(y_test_complain, y_pred_com))
print("Confusion Matrix:", confusion_matrix(y_test_complain, y_pred_com))
print(" ")

# SVC Regression
SVC_complain = SVC(class_weight='balanced')
SVC_complain.fit(X_train_CT_com, y_train_complain)

# Made prediction based on our test set and our linear regression model
y_pred_com = SVC_complain.predict(X_test_CT_com)

# Looked at the metrics of our SVC regression model
accuracy = accuracy_score(y_test_complain, y_pred_com)
print("Testing Set Metrics SVC Regression:")
print("Accuracy:", accuracy)
print("Precision:", precision_score(y_test_complain, y_pred_com, zero_division=␣
↪0))
print("Recall:", recall_score(y_test_complain, y_pred_com))
print("F1 Score:", f1_score(y_test_complain, y_pred_com))
print("Confusion Matrix:", confusion_matrix(y_test_complain, y_pred_com))
print(" ")

# Random Forest Regression
RF_complain = RandomForestClassifier(class_weight='balanced')
RF_complain.fit(X_train_CT_com, y_train_complain)

# Made prediction based on our test set and our linear regression model
y_pred_com = RF_complain.predict(X_test_CT_com)

# Looked at the metrics of our Random Forest regression model
accuracy = accuracy_score(y_test_complain, y_pred_com)
print("Testing Set Metrics Random Forest Regression:")
print("Accuracy:", accuracy)
print("Precision:", precision_score(y_test_complain, y_pred_com, zero_division=␣
↪0))
print("Recall:", recall_score(y_test_complain, y_pred_com))
print("F1 Score:", f1_score(y_test_complain, y_pred_com))
print("Confusion Matrix:", confusion_matrix(y_test_complain, y_pred_com))
print(" ")

# Gradient Boosting Regression
GBR_complain = GradientBoostingClassifier()
GBR_complain.fit(X_train_CT_com, y_train_complain)
```

```
# Made prediction based on our test set and our linear regression model
y_pred_com = GBR_complain.predict(X_test_CT_com)

# Looked at the metrics of our Random Forest regression model
accuracy = accuracy_score(y_test_complain, y_pred_com)
print("Testing Set Metrics Gradient Boosting Regression:")
print("Accuracy:", accuracy)
print("Precision:", precision_score(y_test_complain, y_pred_com, zero_division=␣
↪0))
print("Recall:", recall_score(y_test_complain, y_pred_com))
print("F1 Score:", f1_score(y_test_complain, y_pred_com))
print("Confusion Matrix:", confusion_matrix(y_test_complain, y_pred_com))
```

Our Gradient Boosting Regression Model performed the best with good Accuracy and precision, but still a poor recall and f1 score. The information I took from these models is that Complain is really the biggest predictor of if customer will leave the bank or not and the data isn't easily able to predict the number of years a customer will stay for.

[ ]:
```
# Use the best model (Gradient Boosting without Complain) to flag high-risk␣
↪customers
# Get churn predictions on the test set
y_pred_highrisk = GBR_complain.predict(X_test_CT_com)

# Get probability scores so we can rank customers by churn risk
y_prob_highrisk = GBR_complain.predict_proba(X_test_CT_com)[:, 1]

# Add predictions and risk scores back to the original test set rows
high_risk_df = X_test_complain.copy()
high_risk_df['Predicted_Churn'] = y_pred_highrisk
high_risk_df['Churn_Probability'] = y_prob_highrisk

# Filter to only flagged high-risk customers (predicted to churn)
high_risk_customers = high_risk_df[high_risk_df['Predicted_Churn'] == 1]

# Sort by highest churn probability first
high_risk_customers = high_risk_customers.sort_values('Churn_Probability',␣
↪ascending=False)

print(f"Total customers flagged as high-risk: {len(high_risk_customers)}")
print(f"Top 10 highest risk customers:")
print(high_risk_customers.head(10))
```

[ ]:
```
# Analyze characteristics of high-risk customers to inform retention strategy
print("=== HIGH-RISK CUSTOMER SEGMENT ANALYSIS ===\n")

print("Geography breakdown of high-risk customers:")
```

```python
print(high_risk_customers['Geography'].value_counts())
print(" ")

print("Active vs Inactive members in high-risk group:")
print(high_risk_customers['IsActiveMember'].value_counts())
print(" ")

print("Number of products held by high-risk customers:")
print(high_risk_customers['NumOfProducts'].value_counts())
print(" ")

print("Average balance of high-risk customers vs full test set:")
print(f"High-risk avg balance: ${high_risk_customers['Balance'].mean():,.2f}")
print(f"Full test set avg balance: ${X_test_complain['Balance'].mean():,.2f}")
```